

Kolaysoft CTO Dashboard — 4. Gün Teknik Karar ve Öğrenme Planı

Haftalık Proje Durum Raporlama ve CTO Takip Sistemi

Doküman Bilgisi	Değer
Proje	Kolaysoft CTO Dashboard
Alt Başlık	Haftalık Proje Durum Raporlama ve CTO Takip Sistemi
Doküman Türü	Teknik Karar, Mimari Yaklaşım ve Öğrenme Planı
Tarih	23 Temmuz 2026
Sürüm	1.0
Durum	Teknik planlama aşaması; uygulama geliştirmesi henüz başlamamıştır
Hedef Kitle	CTO, proje yöneticileri, yazılım geliştirme ekibi, teknik mentor ve QA ekibi

İçindekiler

1. Dokümanın Amacı ve Kapsamı
2. Referans Alınan Mevcut Çalışmalar
3. Projenin Mevcut Teknik Durumu
4. Mimari İlkeler ve Karar Verme Yaklaşımı
5. Teknoloji Seçimleri ve Gerekçeleri
6. Genel Sistem Mimarisi
7. Backend Mimari Kararları
8. Frontend Mimari Kararları
9. Veritabanı ve Veri Yönetimi Yaklaşımı
10. API Tasarım Standartları
11. Güvenlik Yaklaşımı
12. Yapılandırma ve Ortam Yönetimi
13. Repository (Depo) ve Sürüm Kontrolü Yaklaşımı
14. Kalite Güvencesi ve Test Stratejisi
15. Dokümantasyon Yaklaşımı
16. Teknik Öğrenme Planı
17. Teknik Riskler ve Azaltma Planları
18. Aşamalı Geliştirme Yol Haritası
19. Başarı Ölçütleri ve Kontrol Noktaları
20. Açık Kararlar ve Varsayımlar
21. Sonuç

1. Dokümanın Amacı ve Kapsamı

Bu dokümanın amacı, Kolaysoft CTO Dashboard projesinin uygulama geliştirme aşamasına geçmeden önce kullanılacak teknoloji, mimari, güvenlik, veri yönetimi, API, repository (depo) ve kalite yaklaşımlarını açıklamaktır. Doküman aynı zamanda 20 günlük staj süresinde ihtiyaç duyulacak teknik öğrenme başlıklarını ve uygulama sırasını tanımlamaktadır.

Bu çalışma bir uygulama tamamlama raporu değildir. Depo incelemesi sırasında backend ve frontend klasörlerinin yalnızca yer tutucu dosyalar içerdiği; veritabanı şeması, örnek veri, uygulama kaynak kodu, testler ve çalışma yapılandırmalarının henüz oluşturulmadığı doğrulanmıştır. Bu nedenle dokümanda teknik bileşenler için “planlanmıştır”, “seçilmiştir” ve “kullanılacaktır” ifadeleri kullanılmaktadır.

Teknik kararların temel hedefleri şunlardır:

- Gün 1–3 arasında belirlenen iş gereksinimlerini uygulanabilir bir teknik yapıya dönüştürmek,
- 20 günlük staj süresine uygun, küçük ve doğrulanabilir geliştirme adımları belirlemek,
- Gereksiz mimari karmaşıklığı önlemek,
- Güvenlik ve yetkilendirme kurallarını yalnızca frontend’e bırakmamak,
- Backend, frontend ve veritabanı katmanlarında tutarlı isimlendirme sağlamak,
- Haftalık raporların izlenebilirliğini ve veri bütünlüğünü korumak,
- Geliştirme sürecinde öğrenilecek teknolojileri doğrudan proje görevleriyle ilişkilendirmek,
- MVP kapsamı dışındaki özelliklerin kontrolsüz biçimde projeye eklenmesini önlemek.

Bu dokümanın kapsamına mikroservis mimarisi, mobil uygulama, gelişmiş dağıtım otomasyonu, üretim izleme altyapısı, Jira/Azure DevOps entegrasyonu, bildirim servisleri, yapay zekâ özellikleri ve dış aktarma özellikleri dahil edilmemiştir.

2. Referans Alınan Mevcut Çalışmalar

Teknik kararlar oluşturulurken depoda bulunan aşağıdaki kaynaklar referans alınmıştır:

- README.md
- docs/analysis/analysis-day1.pdf
- docs/analysis/Day2_Analysis.pdf
- docs/analysis/Day3_Analysis_Document_v1.pdf
- docs/diagrams/UseCase_Diagram.png
- docs/diagrams/ER_Diagram.png
- docs/diagrams/Class_Diagram.png
- docs/diagrams/UserFlow_Diagram.png
- docs/diagrams/Sequence_Diagram.png
- docs/diagrams/SystemArchitecture.png

Gün 1 dokümanında proje problemi, hedef kullanıcılar, temel iş akışı ve ilk modül listesi belirlenmiştir. Gün 2 dokümanında katmanlı mimari, teknoloji ailesi ve ilk UML diyagramları hazırlanmıştır. Gün 3 dokümanında MVP sınırları, roller, kullanıcı hikâyeleri, iş kuralları, doğrulama kuralları, hata senaryoları, API ihtiyaçları ve kabul kriterleri ayrıntılandırılmıştır.

Gün 2 diyagramları ile Gün 3 analiz dokümanı arasında bazı model farklılıkları bulunmaktadır. Gün 2 ER ve sınıf diyagramlarında ayrı `ROLE` ve `PROJECT_ASSIGNMENT` yapıları ile `RISK_ISSUE` modeli yer alırken, Gün 3 analizinde kullanıcı üzerinde rol alanı, proje üzerinde tek yönetici ilişkisi ve

RISK_BLOCKER modeli tanımlanmıştır. MVP'nin daha güncel ve ayrıntılı iş kurallarını içermesi nedeniyle geliştirme sırasında Gün 3 modeli temel kaynak olarak kullanılacaktır. Gün 2 diyagramları kavramsal referans olarak korunacak, ancak ileride güncel modele göre revize edilmeleri gerekecektir.

Benzer şekilde Gün 2 Sequence Diagram üzerinde `/api/auth/login` ve `/api/weekly-reports` yolları görülürken, Gün 3 API planında `/api/v1/auth/login` ve `/api/v1/reports` yolları tanımlanmıştır. API versiyonlama ihtiyacı nedeniyle `/api/v1` standardı seçilmiştir.

3. Projenin Mevcut Teknik Durumu

Depo incelemesine göre proje analiz ve teknik planlama aşamasındadır. Mevcut durum aşağıdaki şekilde değerlendirilmiştir:

- Backend klasöründe çalıştırılabilir Spring Boot projesi bulunmamaktadır.
- Frontend klasöründe React veya Vite projesi bulunmamaktadır.
- `database/schema.sql` ve `database/sample_data.sql` dosyaları boştur.
- `docs/database/database_design.md` dosyası boştur.
- Maven veya Gradle yapılandırması bulunmamaktadır.
- `package.json`, Vite yapılandırması ve frontend kaynak dosyaları bulunmamaktadır.
- Swagger/OpenAPI çalışma yapılandırması bulunmamaktadır.
- JWT kimlik doğrulama ve Spring Security uygulaması bulunmamaktadır.
- Flyway migration dosyaları bulunmamaktadır.
- Otomatik test veya CI/CD yapılandırması bulunmamaktadır.

Bu durum bir hata olarak değil, analiz aşamasından uygulama aşamasına geçiş noktası olarak değerlendirilmiştir. İlk teknik hedef, tüm iş modüllerini aynı anda geliştirmek değil; önce derlenebilen, çalıştırılabilen ve bağlantıları doğrulanabilen bir proje iskeleti oluşturmaktır.

4. Mimari İlkeler ve Karar Verme Yaklaşımı

4.1 Modüler Monolit Yaklaşımı

MVP için modüler monolit yaklaşımı seçilmiştir. Uygulama tek bir Spring Boot backend ve tek bir React frontend olarak geliştirilecektir. Kullanıcı, proje, rapor, iş kalemi, risk/engel ve dashboard alanları mantıksal olarak ayrılacak; ancak ayrı mikroservislere bölünmeyecektir.

Bu kararın gerekçeleri şunlardır:

- 20 günlük staj süresinde mikroservis altyapısı gereksiz operasyonel yük oluşturacaktır.
- İşlem sınırları aynı veritabanı üzerinde daha kolay yönetilecektir.
- Kimlik doğrulama, loglama, hata yönetimi ve yapılandırma tekrar edilmeyecektir.
- Yerel geliştirme ve hata ayıklama daha kolay olacaktır.
- MVP tamamlandıktan sonra gerçek ölçek ve kullanım verilerine göre ayrıştırma kararı alınabilecektir.

4.2 Katmanlı Mimari

Backend için Controller, Service, Repository ve Entity katmanlarından oluşan katmanlı mimari kullanılacaktır. Dış dünya ile veri alışverişi DTO'lar üzerinden yapılacaktır. JPA entity sınıfları controller cevaplarında doğrudan kullanılmayacaktır.

Katmanların sorumlulukları:

- **Controller:** HTTP isteğini kabul edecek, temel istek doğrulamasını başlatacak ve uygun servis metodunu çağıracaktır.
- **Service:** Yetki, sahiplik ve iş kurallarını uygulayacaktır.
- **Repository:** Kalıcılık işlemlerini ve sorguları yönetecektir.
- **Entity:** Veritabanı temsilini taşıyacaktır.
- **DTO:** API istek ve cevap sözleşmesini temsil edecektir.
- **Mapper:** Entity ve DTO dönüşümünü sağlayacaktır.
- **Specification:** Dinamik filtreleri oluşturacaktır.
- **Exception:** Uygulamaya özgü hata tiplerini ve merkezi hata yönetimini içerecektir.

4.3 Sorumlulukların Ayrılması

Frontend rol kontrolü kullanıcı deneyimini iyileştirmek için kullanılacak, fakat güvenlik sınırı olarak kabul edilmeyecektir. Gerçek erişim kontrolü backend tarafında yapılacaktır. Veritabanı kısıtları servis doğrulamalarının yerine geçmeyecek; son savunma katmanı olarak kullanılacaktır.

4.4 Basitlik ve Aşamalı Geliştirme

İlk sürüm için yalnızca gereksinimi karşılayan çözüm geliştirilecektir. Gereksiz genel amaçlı framework'ler, karmaşık state management çözümleri, mesaj kuyrukları, cache sunucuları ve dağıtık sistem desenleri eklenmeyecektir.

Her ana modül şu sırayla ele alınacaktır:

1. Gereksinim ve kabul kriterinin doğrulanması,
2. Veri modeli ve API sözleşmesinin belirlenmesi,
3. Backend uygulaması,
4. Backend testi ve Swagger doğrulaması,
5. Frontend uygulaması,
6. Entegrasyon doğrulaması,
7. Dokümantasyon güncellemesi.

5. Teknoloji Seçimleri ve Gerekçeleri

5.1 Backend Teknolojileri

Java 21

Backend programlama dili olarak Java 21 seçilmiştir. Java 21 uzun süreli destek sürümü olması, güçlü tip sistemi, geniş kurumsal ekosistemi ve Spring Boot ile uyumu nedeniyle kullanılacaktır.

Spring Boot 3

REST API ve uygulama altyapısı için Spring Boot 3 seçilmiştir. Bağımlılık yönetimi, otomatik yapılandırma, web, validation, security ve persistence bileşenlerini ortak bir ekosistem altında sunması tercih nedenidir.

Planlanan temel backend bağımlılıkları:

- Spring Web
- Spring Data JPA
- Spring Security
- Jakarta Validation
- PostgreSQL Driver

- Springdoc OpenAPI
- Flyway Migration
- Lombok
- Spring Boot DevTools
- Spring Boot Test

Maven

Backend build ve bağımlılık yönetimi için Maven kullanılması planlanmıştır. Spring Initializr desteği, standart klasör yapısı ve staj projesinde anlaşılması kolay yaşam döngüsü nedeniyle tercih edilmiştir. Bu seçim uygulama iskeleti oluşturulurken kesinleştirilecektir.

Lombok

Tekrarlayan getter, setter ve constructor kodlarını azaltmak için Lombok kullanılacaktır. Bununla birlikte entity sınıflarında kontrolsüz `@Data` kullanımı yerine ihtiyaca göre `@Getter`, `@Setter` ve constructor anotasyonları tercih edilecektir. Bağımlılık ilişkileri için constructor injection kullanılacaktır.

5.2 Frontend Teknolojileri

React

Kullanıcı arayüzü için React seçilmiştir. Bileşen tabanlı yapı, tekrar kullanılabilir ekran parçaları ve geniş kütüphane desteği tercih nedenidir.

Vite

Frontend geliştirme ve build aracı olarak Vite seçilmiştir. Hızlı geliştirme sunucusu, basit yapılandırma ve React ile uyumu nedeniyle kullanılacaktır.

React Router

Login, dashboard, proje, rapor ve admin ekranları arasındaki istemci tarafı yönlendirme için React Router kullanılacaktır. Korunan rotalar ve rol bazlı rota kontrolleri oluşturulacaktır.

Axios

Backend API çağrıları için Axios seçilmiştir. Tekrar kullanılabilir bir Axios instance oluşturulacak; base URL, JWT header'ı ve merkezi 401/403 yönetimi interceptor'lar üzerinden planlanacaktır.

Material UI

Arayüz bileşenleri ve tutarlı görsel dil için Material UI seçilmiştir. Form, tablo, kart, dialog, navigation ve geri bildirim bileşenlerinde ortak standart sağlayacaktır.

Form Yönetimi

Form karmaşıklığı arttığında React Hook Form kullanılması planlanmıştır. İlk basit ekranlarda gereksiz soyutlama oluşturmayacak şekilde değerlendirilecek; özellikle dinamik Work Item ve Risk/Blocker listeleri içeren haftalık rapor formunda kullanılması tercih edilecektir.

Grafik Kütüphanesi

Chart.js veya Recharts seçenekleri değerlendirilmiştir. Dashboard backend özet endpointleri tamamlanmadan grafik kütüphanesi eklenmeyecektir. İhtiyaç doğrulandığında React bileşen modeliyle uyumu nedeniyle Recharts öncelikli aday olarak değerlendirilmiştir; nihai seçim dashboard geliştirme aşamasında yapılacaktır.

5.3 Veritabanı

İlişkisel veritabanı olarak PostgreSQL seçilmiştir. Proje, kullanıcı, haftalık rapor ve alt kayıtlar arasında güçlü ilişkiler bulunması; benzersizlik kısıtı, foreign key, transaction ve indeks desteği bu kararın temel gerekçeleridir.

5.4 API Dokümantasyonu

REST API'nin etkileşimli olarak incelenmesi ve test edilmesi için Springdoc OpenAPI/Swagger kullanılacaktır. Swagger çıktısı backend kodundaki controller ve DTO tanımlarından üretilecektir.

6. Genel Sistem Mimarisi

Planlanan sistem üç temel çalışma katmanından oluşacaktır:

1. **React Frontend:** Kullanıcı arayüzü, form yönetimi, istemci yönlendirmesi ve API istemcisi,
2. **Spring Boot REST API:** Kimlik doğrulama, yetkilendirme, iş kuralları, veri erişimi ve hata yönetimi,
3. **PostgreSQL:** Kalıcı kullanıcı, proje, rapor, iş kalemi ve risk/engel verileri.

Temel istek akışı aşağıdaki şekilde planlanmıştır:

1. Kullanıcı React uygulamasından giriş isteği gönderecektir.
2. Backend kullanıcı bilgilerini doğrulayacak ve başarılı durumda JWT access token üretecektir.
3. Frontend sonraki isteklerde token'ı `Authorization: Bearer <token>` başlığında gönderecektir.
4. Spring Security token'ı doğrulayacak ve kullanıcı rolünü güvenlik bağlamına yerleştirecektir.
5. Controller isteği ilgili service katmanına aktaracaktır.
6. Service rol, sahiplik ve iş kurallarını kontrol edecektir.
7. Repository PostgreSQL üzerinde gerekli işlemi gerçekleştirecektir.
8. Sonuç DTO ve standart API cevabı olarak frontend'e dönecektir.

Frontend ile backend geliştirme ortamında farklı portlarda çalışacaktır. CORS yalnızca izin verilen frontend origin'i için backend tarafında yapılandırılacaktır. Üretim dağıtım mimarisi MVP uygulama geliştirmesinin dışında tutulmuştur.

7. Backend Mimari Kararları

7.1 Paket Yapısı

Kök paket adı `com.kolaysoft.ctodashboard` olarak seçilmiştir. Aşağıdaki paketler kullanılacaktır:

- `config`
- `controller`
- `dto.request`
- `dto.response`
- `entity`
- `enums`
- `exception`
- `mapper`
- `repository`
- `security`

- `service`
- `service.impl`
- `specification`
- `util`
- `validation`

Paketler yalnızca görünüm oluşturmak amacıyla boş bırakılmayacak; ilgili sorumluluk ortaya çıktıkça sınıflar eklenecektir.

7.2 Entity ve DTO Ayrımı

Entity sınıfları veritabanı ilişkilerini temsil edecektir. API istekleri için request DTO, cevaplar için response DTO kullanılacaktır. Bu yaklaşım:

- `passwordHash` gibi alanların yanlışlıkla dışarı açılmasını önleyecek,
- API sözleşmesini veritabanı modelinden ayırarak,
- Alan bazlı doğrulamayı kolaylaştıracak,
- Gelecekte entity değişikliklerinin istemciyi doğrudan etkilemesini azaltacaktır.

7.3 Servis Katmanı

İş kuralları servis katmanında merkezi olarak uygulanacaktır. Özellikle aşağıdaki kontroller controller veya repository katmanına dağıtılmayacaktır:

- PM'nin yalnızca atanmış projeye rapor oluşturabilmesi,
- Aynı proje/yıl/hafta için tek rapor bulunması,
- `SUBMITTED` raporun PM tarafından değiştirilememesi,
- `YELLOW` veya `RED` raporda en az bir risk/engel bulunması,
- `HIGH` veya `CRITICAL` etki seviyesinde mitigation plan zorunluluğu,
- İlerleme değerlerinin 0–100 arasında olması,
- Hafta numarasının 1–53 arasında olması.

7.4 Repository ve Dinamik Filtreleme

Spring Data JPA repository'leri temel CRUD işlemleri için kullanılacaktır. Dashboard ve rapor filtreleri için JPA Specification veya eşdeğer dinamik sorgu yaklaşımı kullanılacaktır. Her olası filtre kombinasyonu için ayrı repository metodu yazılmayacaktır.

7.5 Hata Yönetimi

`GlobalExceptionHandler` ile merkezi hata yönetimi planlanmıştır. Doğrulama, yetki, kayıt bulunamaması, iş kuralı ihlali, mükerrer kayıt ve beklenmeyen sunucu hataları tutarlı formatta döndürülecektir.

HTTP durum kodları genel olarak şu şekilde kullanılacaktır:

- `200 OK` : Başarılı okuma veya güncelleme,
- `201 Created` : Başarılı kaynak oluşturma,
- `204 No Content` : Uygun olduğu durumda başarılı silme,
- `400 Bad Request` : Doğrulama veya iş kuralı hatası,
- `401 Unauthorized` : Kimlik doğrulama başarısızlığı,
- `403 Forbidden` : Yetkisiz erişim,
- `404 Not Found` : Kaynak bulunamaması,
- `409 Conflict` : Mükerrer haftalık rapor gibi çakışmalar,

- 500 Internal Server Error : Beklenmeyen hata.

Kullanıcıya Türkçe ve anlamlı mesaj gösterilecek; stack trace, SQL detayı veya hassas sistem bilgisi API cevabına yazılmayacaktır.

7.6 Tarih ve Saat Yaklaşımı

Hafta hesaplamalarında ISO-8601 yaklaşımı kullanılacaktır. `weekNumber` alanı 1–53 arasında doğrulanacaktır. `reportDate` , proje başlangıç ve hedef bitiş tarihleri için tarih tipi; oluşturma, güncelleme ve gönderme zamanları için zaman damgası kullanılacaktır.

Sunucu ve veritabanı saat dilimi yaklaşımı geliştirme başlangıcında kesinleştirilecektir. Zaman damgalarının tutarlı saklanması için UTC kullanımı öncelikli yaklaşım olarak değerlendirilecektir.

8. Frontend Mimari Kararları

8.1 Klasör Yapısı

Frontend için aşağıdaki sorumluluk tabanlı yapı kullanılacaktır:

- `src/api`
- `src/assets`
- `src/components/common`
- `src/components/forms`
- `src/components/layout`
- `src/components/dashboard`
- `src/context`
- `src/hooks`
- `src/layouts`
- `src/pages/auth`
- `src/pages/dashboard`
- `src/pages/projects`
- `src/pages/reports`
- `src/pages/admin`
- `src/routes`
- `src/services`
- `src/utils`
- `src/constants`

Sayfa bileşenleri yönlendirme seviyesindeki düzeni yönetecek; tekrar kullanılabilir görsel parçalar `components` altında tutulacaktır. API çağrıları doğrudan her bileşenin içine dağıtılmayacaktır.

8.2 Kimlik Doğrulama Durumu

Kullanıcının oturum bilgisi için bir Auth Context ve ilgili hook planlanmıştır. Context:

- Kullanıcı bilgisi,
- Kullanıcı rolü,
- Giriş/çıkış işlemleri,
- Oturum yüklenme durumu,
- `/auth/me` ile kullanıcı doğrulaması

gibi sorumlulukları yönetecektir.

Token saklama yöntemi güvenlik değerlendirmesiyle kesinleştirilecektir. MVP’de JWT access token kullanılacaktır; ancak tarayıcı depolama tercihinin XSS etkileri dikkate alınacaktır. Frontend tarafından gönderilen rol bilgisine güvenilmeyecek, yetki backend tarafından belirlenecektir.

8.3 Routing

React Router ile aşağıdaki rota sınıfları planlanmıştır:

- Herkese açık login rotası,
- Kimliği doğrulanmış kullanıcı rotaları,
- ADMIN rotaları,
- PROJECT_MANAGER rotaları,
- CTO veya ADMIN dashboard rotaları.

Yetkisiz rota ziyaretinde kullanıcı uygun sayfaya yönlendirilecek ve Türkçe bildirim gösterilecektir. Rota koruması yalnız kullanıcı deneyimi içindir; backend güvenlik kontrolünün yerine geçmeyecektir.

8.4 Axios ve Hata Yönetimi

Tek bir Axios instance kullanılacaktır. Base URL ortam değişkeninden alınacaktır. Request interceptor geçerli JWT’yi istek başlığına ekleyecektir. Response interceptor:

- 401 durumunda oturumun geçersizliğini yönetecek,
- 403 durumunda yetkisiz işlem mesajını gösterecek,
- API hata formatını ortak frontend hata modeline dönüştürecek.

8.5 Form ve Doğrulama

Frontend doğrulaması hızlı kullanıcı geri bildirimi sağlayacaktır; ancak backend doğrulaması zorunlu olmaya devam edecektir. Form mesajları Türkçe olacaktır. Dinamik rapor formunda iş kalemi ve risk/engel listeleri küçük alt bileşenlere ayrılacaktır.

8.6 Dashboard

Dashboard grafikleri, backend summary ve health-distribution endpointleri tamamlandıktan sonra geliştirilecektir. Başlangıçta özelliği tamamlanmış gösterecek sahte veriler veya kalıcı mock metrikler kullanılmayacaktır. Gerekirse yalnız geliştirme amaçlı geçici mock veriler açıkça işaretlenecek ve gerçek entegrasyon tamamlanmadan özellik tamamlanmış kabul edilmeyecektir.

9. Veritabanı ve Veri Yönetimi Yaklaşımı

9.1 Temel Veri Modeli

MVP için Gün 3 analizindeki aşağıdaki beş temel entity esas alınacaktır:

- User
- Project
- WeeklyReport
- WorkItem
- RiskBlocker

Roller ADMIN, PROJECT_MANAGER ve CTO sabit değerleriyle temsil edilecektir. Bir proje MVP’de bir Project Manager’a atanacaktır; bir Project Manager birden fazla proje yönetebilecektir.

9.2 İsimlendirme

Veritabanında tablo ve kolon isimleri `snake_case` olacaktır. Java tarafında sınıf ve alanlar İngilizce `PascalCase` ve `camelCase` kurallarına göre adlandırılacaktır.

Örnek eşleşmeler:

- `WeeklyReport` → `weekly_reports`
- `weekNumber` → `week_number`
- `overallHealth` → `overall_health`
- `passwordHash` → `password_hash`

9.3 Veritabanı Kısıtları

Veri bütünlüğü için primary key, foreign key, not-null ve unique constraint yapıları kullanılacaktır. Aynı proje için aynı yıl ve hafta numarasında birden fazla rapor oluşturulmasını önlemek amacıyla aşağıdaki birleşik benzersizlik kısıtı planlanmıştır:

```
UNIQUE(project_id, year, week_number)
```

Bu kısıt servis seviyesindeki kontrolü destekleyecektir. Eşzamanlı isteklerde veritabanı son güvence katmanı olacaktır.

9.4 İndeksleme

Sık filtrelenecek alanlarda ölçülü indeks kullanılması planlanmıştır:

- Kullanıcı e-postası,
- Proje kodu,
- Proje yöneticisi,
- Proje durumu,
- Rapor projesi,
- Rapor yılı ve hafta numarası,
- Rapor durumu ve sağlık değeri,
- Risk/engel etki seviyesi ve durumu.

Her kolona otomatik indeks eklenmeyecek; sorgu ihtiyaçlarına göre değerlendirme yapılacaktır.

9.5 Migration Yönetimi

Şema değişiklikleri için Flyway seçilmiştir. JPA'nın üretim benzeri ortamlarda şemayı kontrolsüz değiştirmesine izin verilmeyecektir. Başlangıç migration sırası aşağıdaki şekilde planlanmıştır:

1. `V1__create_users_table.sql`
2. `V2__create_projects_table.sql`
3. `V3__create_weekly_reports_table.sql`
4. `V4__create_work_items_table.sql`
5. `V5__create_risk_blockers_table.sql`
6. `V6__create_indexes_and_constraints.sql`
7. `V7__insert_initial_admin.sql`

Başlangıç admin kullanıcısı eklenirken düz metin parola tutulmayacaktır. Parola hash'inin güvenli şekilde nasıl sağlanacağı uygulama başlamadan önce netleştirilecek; gerçek sırlar depoya yazılmayacaktır.

9.6 Örnek Veri

`sample_data.sql` yalnız yerel geliştirme amacıyla kullanılacaktır. Gerçek kişi bilgileri, gerçek parolalar veya hassas şirket verileri eklenmeyecektir. Örnek veri yükleme yaklaşımı profil bazlı planlanacaktır.

10. API Tasarım Standartları

10.1 Temel Yol ve Versiyonlama

Tüm uygulama endpointleri `/api/v1` temel yolu altında sunulacaktır. Böylece gelecekte sözleşme değişikliği gerektiğinde yeni API sürümü kontrollü biçimde oluşturulabilecektir.

Planlanan ana kaynak yolları:

- `/api/v1/auth`
- `/api/v1/users`
- `/api/v1/projects`
- `/api/v1/reports`
- `/api/v1/dashboard`

Endpoint ve JSON alan adları İngilizce olacaktır. Kullanıcı mesajları Türkçe olacaktır.

10.2 Kaynak Odaklı Tasarım

HTTP metotları amaçlarına uygun kullanılacaktır:

- GET : Veri okuma,
- POST : Kaynak oluşturma veya açık bir komut işlemi,
- PUT : Kaynağın güncellenmesi,
- PATCH : Kısmi durum veya ilişki güncellemesi,
- DELETE : Yetkili silme işlemi.

Rapor gönderme işlemi basit alan güncellemesinden daha fazla iş kuralı içerdiği için POST `/api/v1/reports/{id}/submit` komut endpointi olarak kullanılacaktır.

10.3 Cevap Standardı

Başarılı cevaplarda ihtiyaç olduğunda aşağıdaki ortak yapı kullanılacaktır:

- `success`
- `message`
- `data`

Doğrulama hatalarında:

- `success`
- `message`
- `errors`

alanları kullanılacaktır. `errors` alanı, form alanı ile Türkçe hata mesajını eşleştirecektir.

10.4 Filtreleme ve Sayfalama

Proje ve rapor filtreleri query parameter olarak taşınacaktır. Planlanan filtreler:

- `projectId`
- `managerId`

- `year`
- `weekNumber`
- `status`
- `health`
- `riskLevel`

Liste veri hacmi arttığında sayfalama için `page`, `size` ve `sort` parametreleri kullanılacaktır. Sayfalamanın ilk liste endpointlerinden itibaren uygulanıp uygulanmayacağı geliştirme sırasında veri hacmi ve süreye göre kesinleştirilecektir.

10.5 OpenAPI

Controller endpointleri, DTO alanları, doğrulama kuralları, güvenlik şeması ve olası hata cevapları OpenAPI üzerinde belgelenecektir. Swagger arayüzünün üretim ortamında açık olup olmayacağı dağıtım aşamasında ayrıca kararlaştırılacaktır.

11. Güvenlik Yaklaşımı

11.1 Kimlik Doğrulama

Kullanıcılar e-posta ve parola ile giriş yapacaktır. Parolalar BCrypt ile hash'lenmiş olarak saklanacaktır. Başarılı girişte JWT access token üretilecektir. JWT içinde yalnız gerekli kimlik ve yetki bilgileri bulunacaktır.

Parola veya JWT değerleri loglanmayacaktır. `passwordHash` API cevaplarında yer almayacaktır.

11.2 Yetkilendirme

Spring Security ile endpoint seviyesinde rol tabanlı yetkilendirme uygulanacaktır. Gerekli durumlarda `@PreAuthorize` gibi metot seviyesinde güvenlik kontrolleri kullanılacaktır.

Planlanan genel yetki sınırları:

- Authentication endpointleri herkese açık olacaktır.
- Kullanıcı oluşturma ve rol yönetimi yalnız ADMIN tarafından yapılacaktır.
- Proje oluşturma, güncelleme ve yönetici atama yalnız ADMIN tarafından yapılacaktır.
- Rapor oluşturma, taslak güncelleme ve gönderme PROJECT_MANAGER tarafından yapılacaktır.
- CTO tüm proje ve raporları okuyabilecek, dashboard verilerine erişebilecektir.
- ADMIN genel yönetim ve görüntüleme yetkilerine sahip olacaktır.
- PROJECT_MANAGER yalnız kendisine atanmış projeleri ve bu projelerin raporlarını görebilecektir.

Rol kontrolüne ek olarak kaynak sahipliği kontrolü yapılacaktır. PROJECT_MANAGER rolüne sahip olmak, tüm projeler üzerinde işlem yapmak için yeterli kabul edilmeyecektir.

11.3 Input Validation

Jakarta Validation ile DTO alanlarında zorunluluk, uzunluk, format ve aralık kontrolleri uygulanacaktır. Birden fazla alan veya alt kayıt gerektiren kurallar servis veya özel validation bileşenlerinde ele alınacaktır.

11.4 CORS

CORS izinleri React frontend origin'i ile sınırlandırılacaktır. Tüm origin'lere kontrolsüz izin verilmesi planlanmamaktadır. İzin verilen metot ve header'lar ihtiyaçla sınırlı tutulacaktır.

11.5 Hassas Yapılandırma

Veritabanı parolası, JWT secret ve benzeri değerler kaynak koda yazılmayacaktır. Ortam değişkenleri veya yerel olarak dışarıda tutulan yapılandırma dosyaları kullanılacaktır. Örnek yapılandırma gerçek secret içermeyen belgelenecektir.

11.6 Güvenlik Sınırları

MVP kapsamında refresh token, parola sıfırlama, MFA, rate limiting ve kapsamlı audit log özellikleri henüz kesinleştirilmemiştir. Bu özellikler mevcut kapsamda tamamlanmış kabul edilmeyecek; ihtiyaç halinde ayrı karar olarak ele alınacaktır.

12. Yapılandırma ve Ortam Yönetimi

Backend yapılandırmasının profil bazlı yönetilmesi planlanmıştır:

- `default` : Ortak ayarlar,
- `dev` : Yerel geliştirme ayarları,
- `test` : Otomatik test ayarları.

Veritabanı bağlantısı için URL, kullanıcı adı ve parola ortam değişkenlerinden alınacaktır. Frontend backend adresi Vite ortam değişkeni üzerinden tanımlanacaktır.

Depoya gerçek `.env` dosyası eklenmeyecektir. Gerekli değişkenlerin adlarını gösteren, secret içermeyen örnek dosya oluşturulması uygulama iskeleti aşamasında değerlendirilecektir.

İlk geliştirme aşamasında Docker kullanılmayacaktır; çünkü mevcut proje planında Docker açıkça talep edilmemiştir. Yerel Java, Node.js ve PostgreSQL kurulumlarıyla çalıştırma yönergeleri README içinde belgelenecektir. Docker gereksinimi daha sonra ayrıca onaylanırsa kapsam değişikliği olarak ele alınacaktır.

13. Repository (Depo) ve Sürüm Kontrolü Yaklaşımı

Planlanan üst seviye yapı korunacaktır:

- `backend`
- `frontend`
- `database`
- `docs`
- `.gitignore`
- `README.md`

Mevcut analiz PDF'leri ve diyagramlar silinmeyecek veya üzerine yazılmayacaktır. Yeni belgeler uygun klasörlere eklenecektir.

13.1 Commit Yaklaşımı

Commitler küçük, anlamlı ve tek sorumluluklu olacaktır. Örnek commit mesajları:

- `docs: add day 4 technical decision document`
- `chore: initialize backend project structure`
- `feat: add backend health endpoint`
- `chore: initialize React frontend structure`
- `feat: connect frontend to backend health API`

- feat: add JWT authentication

Dokümantasyon, backend ve frontend iskeletleri mümkün olduğunca ayrı commitler halinde tutulacaktır. Destructive Git komutları kullanılmayacaktır.

13.2 İkili (Binary) Dokümanlar

Mevcut PDF ve PNG dosyaları korunacaktır. İleride diyagram revizyonu yapılırsa düzenlenebilir .drawio kaynaklarının da depoya eklenmesi planlanmalıdır. Böylece yalnız görsel çıktı değil, değiştirilebilir kaynak da sürüm kontrolü altında tutulabilecektir.

13.3 Branch Yaklaşımı

Staj projesinin ekip çalışma yöntemi doğrulanmadan kesin bir branch stratejisi dayatılmayacaktır. Küçük özellik branch'leri ve pull request yaklaşımı tercih edilebilir. Ana dalın her önemli adımda derlenebilir tutulması hedeflenecektir.

14. Kalite Güvencesi ve Test Stratejisi

14.1 Backend Testleri

Backend için aşağıdaki test seviyeleri planlanmıştır:

- Servis iş kuralları için unit test,
- Controller ve güvenlik davranışları için web katmanı testi,
- Repository sorguları için veri katmanı testi,
- Kritik akışlar için integration test.

Öncelikli test senaryoları:

- PM'nin atanmadığı projeye rapor oluşturmaması,
- Aynı proje/yıl/hafta için ikinci raporun engellenmesi,
- SUBMITTED raporun PM tarafından güncellenememesi,
- YELLOW veya RED sağlık durumunda risk/engel zorunluluğu,
- HIGH veya CRITICAL risk için mitigation plan zorunluluğu,
- CTO'nun okuma erişimi,
- ADMIN dışındaki rollerin kullanıcı ve proje yönetememesi.

14.2 Frontend Testleri

Frontend test araçlarının nihai seçimi frontend iskeleti aşamasında yapılacaktır. Kritik form doğrulamaları, route guard davranışı ve API hata gösterimi test edilmesi planlanan alanlardır. Süre kısıtı nedeniyle yalnız görsel snapshot testlerine güvenilmeyecektir.

14.3 Manuel Kabul Testleri

Gün 3 kabul kriterleri Given–When–Then biçiminde manuel ve otomatik testlere temel olacaktır. Swagger üzerinden API akışları, tarayıcı üzerinden rol bazlı ekran erişimi kontrol edilecektir.

14.4 Kod Kalitesi

Clean Code ve SOLID ilkeleri uygulanacaktır. Küçük ve odaklı sınıflar, constructor injection, anlamlı İngilizce isimler ve merkezi hata yönetimi kullanılacaktır. İş kuralları tekrar edilmeyecektir.

15. Dokümantasyon Yaklaşımı

Dokümantasyon yaşayan bir proje bileşeni olarak ele alınacaktır. Ancak tamamlanmamış bir özellik belgelerde tamamlanmış gibi gösterilmeyecektir.

Planlanan dokümantasyon çıktıları:

- README kurulum ve çalışma adımları,
- Günlük/haftalık staj raporları,
- Güncel mimari ve veri modeli açıklamaları,
- Swagger/OpenAPI çıktısı,
- Kabul kriterleri ve test sonuçları,
- Gerektiğinde ekran görüntüleri.

Doküman dili Türkçe olacaktır. Kod, sınıf, değişken, paket ve endpoint adları İngilizce tutulacaktır.

16. Teknik Öğrenme Planı

Öğrenme planı, teorik konuları doğrudan proje çıktılarıyla ilişkilendirecek şekilde hazırlanmıştır.

16.1 Java 21 ve Spring Boot Temelleri

Öğrenilecek konular:

- Spring Boot proje yapısı,
- Dependency injection ve bean yaşam döngüsü,
- Constructor injection,
- REST controller ve HTTP durum kodları,
- DTO kullanımı,
- Jakarta Validation,
- Configuration ve profile yönetimi.

Planlanan uygulama çıktıları:

- Derlenebilir backend iskeleti,
- Health endpoint,
- Standart cevap ve hata yapısı.

16.2 Spring Data JPA ve PostgreSQL

Öğrenilecek konular:

- Entity mapping,
- One-to-many ve many-to-one ilişkileri,
- Fetch yaklaşımı,
- Repository yapısı,
- Transaction yönetimi,
- Unique constraint ve indeksler,
- Flyway migration yaşam döngüsü,
- JPA Specification.

Planlanan uygulama çıktıları:

- User, Project, WeeklyReport, WorkItem ve RiskBlocker şeması,

- Migration dosyaları,
- Dinamik filtreleme altyapısı.

16.3 Spring Security ve JWT

Öğrenilecek konular:

- Authentication ve authorization farkı,
- Security filter chain,
- BCrypt,
- JWT üretme ve doğrulama,
- Security context,
- Role ve resource ownership kontrolleri,
- 401 ve 403 ayrımı,
- CORS.

Planlanan uygulama çıktıları:

- Login API,
- /auth/me ,
- Rol bazlı endpoint koruması,
- PM proje sahipliği kontrolü.

16.4 React ve Vite

Öğrenilecek konular:

- React component modeli,
- Props ve state,
- Hook kullanımı,
- Context API,
- React Router,
- Form yönetimi,
- Material UI tema ve bileşenleri,
- Ortam değişkenleri.

Planlanan uygulama çıktıları:

- Login sayfası,
- Temel layout,
- Rol bazlı rotalar,
- Proje ve rapor sayfaları.

16.5 Axios ve Entegrasyon

Öğrenilecek konular:

- Axios instance,
- Request/response interceptor,
- JWT header yönetimi,
- API hata modeli,
- Loading ve error state,
- Frontend–backend CORS iletişimi.

Planlanan uygulama çıktıları:

- Health bağlantı göstergesi,
- Merkezi API istemcisi,
- Login ve korunan istekler.

16.6 Test ve Hata Ayıklama

Öğrenilecek konular:

- JUnit 5,
- Mockito,
- Spring Boot integration test,
- Test verisi hazırlama,
- Tarayıcı geliştirici araçları,
- Network istek inceleme,
- Swagger ile API testi.

Planlanan uygulama çıktıları:

- Kritik iş kurallarının testleri,
- Kabul kriteri doğrulama kayıtları.

16.7 Öğrenme Yöntemi

Her konu için aşağıdaki döngü kullanılacaktır:

1. Resmî dokümandan temel kavramın okunması,
2. Projede küçük bir örneğin uygulanması,
3. Derleme veya test ile doğrulama,
4. Hata ve öğrenilen noktaların kaydedilmesi,
5. Sonraki modülde aynı yaklaşımın tekrar kullanılması.

Kopyalanan, açıklanamayan veya derlenmeyen kod proje çıktısı olarak kabul edilmeyecektir.

17. Teknik Riskler ve Azaltma Planları

17.1 Analiz Modelleri Arasındaki Tutarsızlık

Risk: Gün 2 diyagramları ile Gün 3 entity ve API modeli farklıdır.

Etki: Yanlış tablo, ilişki veya endpoint geliştirilmesi.

Azaltma: Gün 3 ayrıntılı MVP analizi birincil referans olarak kullanılacak; diyagramlar kodlama öncesi kontrol listesiyle karşılaştırılacaktır.

17.2 Zaman Kısıtı

Risk: 20 günlük sürede tüm MVP işlevlerinin aynı anda ele alınması yarım modüller oluşturabilir.

Etki: Çalışmayan entegrasyon ve test edilemeyen özellikler.

Azaltma: İskelet, auth, proje, rapor ve dashboard sırası izlenecek; her faz çalışır durumda kapatılacaktır.

17.3 Yetkilendirme Hataları

Risk: Yalnız role bakılması, PM'nin başka projelere erişmesine yol açabilir.

Etki: Yetkisiz veri erişimi.

Azaltma: Role ek olarak proje sahipliği servis katmanında ve testlerde doğrulanacaktır.

17.4 JWT Saklama ve Secret Yönetimi

Risk: Token veya secret'ın güvensiz saklanması.

Etki: Oturum ele geçirme veya kimlik doğrulama sisteminin zayıflaması.

Azaltma: Secret depoya yazılmayacak; token saklama yaklaşımı XSS riski dikkate alınarak kesinleştirilecektir.

17.5 Veri Bütünlüğü

Risk: Eşzamanlı isteklerde aynı haftaya ait birden fazla rapor oluşması.

Etki: Resmî haftalık raporun belirsizleşmesi.

Azaltma: Servis kontrolüne ek olarak birleşik benzersizlik kısıtı kullanılacaktır.

17.6 Entity Döngüleri ve Hassas Alanların Açılması

Risk: JPA entity'lerinin doğrudan JSON olarak döndürülmesi döngüsel serileştirme ve veri sızıntısı oluşturabilir.

Etki: API hataları veya `passwordHash` gibi alanların açılması.

Azaltma: Controller'lar yalnız DTO döndürecektir.

17.7 Frontend ve Backend Doğrulama Farkları

Risk: İki katmanda farklı doğrulama kuralları uygulanması.

Etki: Kullanıcı deneyimi ve API davranışı arasında tutarsızlık.

Azaltma: Backend kuralları kaynak kabul edilecek; frontend aynı mesaj ve sınırlarla uyumlu tutulacaktır.

17.8 Büyük Doküman Dosyaları ve Kaynak Eksikliği

Risk: Diyagramların yalnız büyük PNG çıktılarıyla tutulması.

Etki: Versiyon farklarının izlenememesi ve güncelleme zorluğu.

Azaltma: İleride `.drawio` kaynakları eklenecek; mevcut dosyalar silinmeyecektir.

17.9 Dashboard Sorgu Performansı

Risk: Çok sayıda proje ve raporda kontrolsüz sorguların yavaşlaması.

Etki: CTO dashboard yükleme süresinin artması.

Azaltma: Özet endpointleri özel sorgular, projection ve gerekli indekslerle tasarlanacaktır. Erken aşamada gereksiz optimizasyon yapılmayacaktır.

18. Aşamalı Geliştirme Yol Haritası

Faz 1 — Repository (Depo) ve Karar Doğrulama

- Mevcut dokümanlar korunacaktır.
- Teknik kararlar ve açık noktalar doğrulanacaktır.
- Gün 3 modeli geliştirme kaynağı olarak sabitlenecektir.

Faz 2 — Backend İskeleti

- Spring Boot projesi oluşturulacaktır.

- Java 21 ve gerekli bağımlılıklar yapılandırılacaktır.
- Temel paket yapısı hazırlanacaktır.
- PostgreSQL bağlantı ayarları planlanacaktır.
- Flyway ve Swagger tabanı eklenecektir.
- Health endpoint oluşturulacaktır.
- Backend derlemesi doğrulanacaktır.

Faz 3 — Frontend İskeleti

- React ve Vite projesi oluşturulacaktır.
- React Router, Axios ve Material UI eklenecektir.
- Temel layout ve boş rota sayfaları hazırlanacaktır.
- Frontend build ve çalışma komutları doğrulanacaktır.

Faz 4 — Temel Entegrasyon

- Backend CORS ayarı yapılacaktır.
- Axios base URL yapılandırılacaktır.
- Frontend health endpointi çağıracaktır.
- Bağlantı durumu ekranda gösterilecektir.

Faz 5 — Authentication ve Authorization

- User entity ve Role enum oluşturulacaktır.
- BCrypt ve JWT uygulanacaktır.
- Login ve `/auth/me` endpointleri hazırlanacaktır.
- Auth Context ve protected route yapısı eklenecektir.
- Rol bazlı erişim test edilecektir.

Faz 6 — Proje Yönetimi

- Project entity, DTO, service ve controller geliştirilecektir.
- ADMIN proje CRUD ve manager assignment işlemleri hazırlanacaktır.
- PM ve CTO için role uygun okuma davranışları uygulanacaktır.
- Proje liste ve detay ekranları geliştirilecektir.

Faz 7 — Haftalık Rapor Yönetimi

- WeeklyReport, WorkItem ve RiskBlocker modelleri geliştirilecektir.
- Draft ve submit akışı uygulanacaktır.
- İş kuralları ve benzersizlik kısıtı eklenecektir.
- Haftalık rapor formu ve detay ekranı hazırlanacaktır.

Faz 8 — CTO Dashboard

- Summary, health distribution, critical risk ve latest report endpointleri geliştirilecektir.
- Filtreler eklenecektir.
- Backend verileri doğrulandıktan sonra dashboard kartları ve grafikler hazırlanacaktır.

Faz 9 — Test ve Dokümantasyon

- Kritik unit ve integration testler tamamlanacaktır.

- Kabul kriterleri çalıştırılacaktır.
- Swagger gözden geçirilecektir.
- README çalışma yönergeleri güncellenecektir.
- Uygun ekran görüntüleri eklenecektir.

Bu fazların hiçbiri mevcut dokümanın yazıldığı tarihte tamamlanmış kabul edilmemektedir.

19. Başarı Ölçütleri ve Kontrol Noktaları

Her faz sonunda aşağıdaki kontrol noktaları uygulanacaktır:

- Proje ilgili build komutuyla başarıyla derlenebiliyor mu?
- Uygulama beklenen ortam değişkenleriyle çalışabiliyor mu?
- Yeni endpoint Swagger üzerinde görülebiliyor mu?
- Yetkisiz erişim backend tarafından engelleniyor mu?
- Hata cevapları standart ve Türkçe mi?
- İlgili iş kuralı otomatik veya manuel testle doğrulandı mı?
- Mevcut dokümanlar korunuyor mu?
- Değişen dosyalar ve test komutları raporlandı mı?
- Özellik gerçekten tamamlanmadan tamamlanmış olarak işaretlenmiş mi?

MVP teknik başarısı yalnız ekranların görünmesiyle ölçülmeyecektir. Veri kalıcılığı, rol bazlı erişim, iş kuralları, hata mesajları ve tekrar çalıştırılabilir kurulum birlikte değerlendirilecektir.

20. Açık Kararlar ve Varsayımlar

Aşağıdaki noktalar henüz uygulama ile doğrulanmamış veya paydaş kararı gerektiren varsayımlar olarak bırakılmıştır:

1. Maven kullanımı planlanmıştır; backend iskeleti oluşturulmadan önce kesinleştirilecektir.
2. Gün 3 modeli birincil referans kabul edilmiştir; eski diyagramların ne zaman güncelleneceği ayrıca planlanacaktır.
3. Kurumsal e-posta için yalnız @kolaysoft.com.tr domain zorunluluğu Gün 3 dokümanında yer almaktadır; nihai kural paydaş tarafından doğrulanmalıdır.
4. Haftalık raporların Cuma günü saat 17.00'ye kadar girilmesi kuralının yalnız bilgilendirme mi, yoksa sistem tarafından engellenecek bir kural mı olduğu netleştirilmelidir.
5. JWT access token'ın tarayıcıda saklanma yöntemi güvenlik değerlendirmesi sonrası kesinleştirilecektir.
6. Refresh token ve parola sıfırlama MVP kapsamında tanımlanmamıştır.
7. Başlangıç admin hesabının güvenli oluşturulma yöntemi kesinleştirilecektir; düz metin parola kullanılmayacaktır.
8. UTC ve yerel saat dilimi kullanımının sınırları geliştirme başlangıcında belirlenecektir.
9. Liste endpointlerinde sayfalamanın ilk sürümden itibaren zorunlu olup olmadığı doğrulanacaktır.
10. Dashboard grafik kütüphanesi backend endpointleri tamamlandıktan sonra seçilecektir.
11. Frontend test kütüphanesi frontend iskeleti sırasında kesinleştirilecektir.
12. Docker, CI/CD ve üretim izleme altyapısı MVP kapsamı dışında tutulmuştur.
13. ADMIN'in taslak rapor silme veya iptal etme davranışının ayrıntılı kuralı netleştirilecektir.

14. Submit edilmiş raporun ADMIN tarafından tekrar açılması MVP için planlanmamıştır; rapor kilitli kalacaktır.
15. docs/diagrams altındaki kaynak .drawio dosyalarının mevcut olup olmadığı ve depoya eklenip eklenmeyeceği doğrulanmalıdır.

Bu açık noktalar çözülmeden ilgili özelliğin davranışı varsayımla genişletilmeyecektir.

21. Sonuç

Kolaysoft CTO Dashboard için Java 21, Spring Boot 3, React, Vite ve PostgreSQL tabanlı modüler monolit ve katmanlı mimari seçilmiştir. REST API, JWT, Spring Security, DTO, merkezi hata yönetimi, Flyway migration, rol ve kaynak sahipliği kontrolleri temel teknik yaklaşım olarak planlanmıştır.

Frontend tarafında React Router, Axios ve Material UI kullanılacaktır. Backend ile frontend küçük ve doğrulanabilir adımlarla geliştirilecek; dashboard grafikleri gerçek backend özet endpointleri hazır olmadan tamamlanmış kabul edilmeyecektir.

Veri modelinde Gün 3 analiz dokümanı esas alınmış; Gün 2 diyagramlarıyla olan farklılıklar açıkça kaydedilmiştir. Mevcut analiz belgeleri ve depo yapısı korunacaktır. Geliştirme; çalışan iskelet, authentication, proje yönetimi, haftalık raporlar, CTO dashboard ve test/dokümantasyon sırasıyla ilerleyecektir.

Bu doküman 4. gün sonunda teknik yönü ve öğrenme planını belirlemektedir. Uygulama özelliklerinin geliştirildiğini veya tamamlandığını beyan etmemektedir.